

Projet de théorie des graphes 2025-2026

Préambule :

Avec le développement d'IA génératives toujours plus performantes, il n'est plus envisageable de proposer des projets basés essentiellement sur la production de code source et d'un rapport. Dans ce projet, l'accent est dès lors placé sur la compréhension des concepts et vos capacités à communiquer.

Extrait de l'engagement pédagogique : "... l'évaluation portera essentiellement sur la compréhension des concepts de théorie des graphes utilisés, des principes algorithmiques mis en oeuvre et sur l'implication de l'étudiant.e dans le projet, plutôt que sur la seule production de code."

Consignes :

Le **code source** sera fourni en C ou Python 3 "standard" et utilisera les bibliothèques usuelles. Il sera correct, efficace et intelligible. Si des options particulières sont nécessaires à la compilation, il est indispensable de fournir un Makefile. En vous assurant de la compatibilité et de la disponibilité, vous pouvez utiliser des libraires ou modules additionnels. Cependant, vous devez coder les parties principales inhérentes au projet et ne pas utiliser une librairie prête à l'emploi. L'utilisation de NetworkX pour les manipulations basiques de graphes est autorisée. Par exemple, s'il vous est demandé explicitement de coder la recherche d'un plus court chemin, il ne faut pas prendre la fonction `shortest_path` toute faite dans NetworkX. Un code ne compilant pas entraîne une note de zéro au projet.

- Le choix des noms de variables et de sous-programmes doit faciliter la lecture et la compréhension du code.
- L'emploi de commentaires judicieux est indispensable : entrée/sortie des différents sous-programmes, points clés à commenter, boucles, *etc.*
- Enfin, l'indentation et l'aération doivent aussi faciliter la lecture de votre code en identifiant les principaux blocs.
- Un code peu clair, même si le programme "tourne", sera pénalisé.

Le code sera accompagné d'un **rapport** (max. 6 pages de texte) au format pdf. Le rapport ne doit pas être un copier-coller du code source. Il doit décrire la stratégie utilisée, les choix opérés, les grandes étapes des différentes procédures ou fonctions. Il pourra aussi présenter les difficultés/challenges rencontrés en cours d'élaboration ou reprendre certains résultats expérimentaux (benchmarking sur des exemples types ou générés aléatoirement). Il doit aussi servir de manuel d'installation et de mode d'emploi. Citez vos sources (construire une petite bibliographie/sitographie) et détaillez votre contribution par rapport à ces références. N'hésitez pas à consulter et comparer plusieurs sources pour sélectionner les plus pertinentes. Il est dommage de voir que des étudiants se contentent souvent d'une unique référence Wikipedia ou des sources proposées, faites preuve d'esprit critique.

Groupe de maximum 2 étudiants. Pas de groupe de 3 étudiants. Si nombre impair d'étudiants, il y aura au moins un travail réalisé en solo. Si un accord entre les étudiants n'est pas trouvé, le titulaire procédera à un tirage au sort des groupes.

Le plagiat est interdit : il est interdit d'échanger des solutions complètes, partielles ou de les récupérer sur Internet. Citer vos sources ! Néanmoins, vous êtes autorisés à discuter entre groupes. Mentionner les sources utilisées/consultées.

Timing :

- lundi 6 octobre 2025 : choix individuel des modalités d'examen, répartition en groupes et choix du sujet. Les délégués créeront un fichier partagé (avec 3 colonnes : sujet 1, sujet 2 ou pas de projet).
- jeudi 11 décembre 2025 (minuit) : dépôt du code et du rapport sous forme d'une archive envoyée par mail au titulaire du cours. Cette archive contiendra deux répertoires, un pour le code à compiler, l'autre pour le rapport.
- lundi 15 décembre 2025 - ordre de passage à déterminer : présentation orale.
- janvier 2026 : examen écrit (commun pour tous).

Défense orale :

Pour la **présentation orale**, il est conseillé que les étudiants utilisent un ordinateur pour faire tourner leur programme, pour présenter leur code, pour s'aider d'un support type "power point". Un projecteur vidéo est à disposition. Le but de cette présentation est de fournir une synthèse et d'expliquer les concepts utilisés en maximum 10 minutes.

- Présenter, de façon pédagogique, la théorie nécessaire et le problème posé puis discuter de l'implémentation de votre solution.
- Soignez votre présentation, ne supposez pas que l'auditeur connaisse le problème (imaginez faire la présentation à un "client"). Vous serez évalué.e sur votre capacité à présenter un concept technique de façon claire et concise.
- Respectez le timing.

La présentation sera suivie par une **séance de questions**. Le but de ces questions est de déterminer la contribution et l'implication de chacun. Ces questions pourront être "pointues" : rôle d'une variable dans le code, principe de fonctionnement d'une fonction particulière, détails d'implémentation, etc.

Les **deux projets** ci-dessous sont “génériques”, il est loisible à chaque groupe d'aller plus loin, d'adapter et de développer plus en avant les fonctionnalités de son code (par exemple, meilleure gestion des entrées/sorties, optimisation des structures de données, fournir des exemples “types” dans un fichier, etc.). Le principe est de proposer une thématique générale qu'il vous est loisible de développer. Soyez originaux.

Sujet 1 : Flot maximal dans un réseau

On modélise un réseau de transport par un graphe orienté, dans lequel chaque arête représente un canal de communication (route, canalisation, câble, etc.) et possède une capacité maximale, c'est-à-dire la quantité maximale de flux qu'elle peut transporter. Deux points particuliers sont distingués : la source, d'où part tout le flux, et le puits, où le flux doit arriver. Le problème du flot maximal consiste à déterminer quelle est la quantité maximale de flux que l'on peut acheminer de la source vers le puits en respectant les limites de capacité des arcs et en assurant que ce qui entre dans un sommet intermédiaire en ressort.

- Implémenter un algorithme permettant de résoudre ce problème. Votre programme doit prendre en entrée, un fichier standardisé. Chaque ligne du fichier contient 3 nombres séparés par un espace, par exemple : 2 8 10. Les deux premiers nombres décrivent l'arc (indices de l'origine 2 et de la destination 8 de l'arc), le dernier représente la capacité maximale 10 de l'arc. En particulier, vous devez vous assurer que le fichier fourni est valide et, dans le cas contraire, informer l'utilisateur.
- Votre programme fournira en sortie un fichier du même type avec, sur chaque ligne 3 nombres séparés par un espace. Ces 3 nombres codent la solution trouvée : les arcs du graphe et la capacité réellement utilisée pour transporter le flux. Ces entrées/sorties standards permettent de tester votre code de façon automatique.
- Enfin, il vous est demandé de réfléchir à l'application de ces concepts pour obtenir le théorème de Menger à propos de chemins disjoints dans un graphe.
- Pistes d'améliorations : visualisation graphique, animation montrant l'algorithme en action, recherche d'applications variées, développement d'un jeu de données pertinent, etc.

Sources conseillées :

- [Combinatorial optimization](#), par B. Korte et J. Vygen (chapitre 8) ; accessible depuis une machine connectée au Réseau ULiège.
- L'[article original](#) de Ford et Fulkerson ; accessible depuis une machine connectée au Réseau ULiège.
- Une application dans le sport, [MIT assignment](#) ; accessible depuis une machine connectée au Réseau ULiège.

Sujet 2 : Sous-arbre couvrant de poids minimal

On considère un graphe non orienté où chaque arête est pondérée. Le problème du sous-arbre couvrant de poids minimal (MST Minimum Spanning Tree) consiste à sélectionner un ensemble d'arêtes qui relie tous les sommets du graphe, sans former de cycle, de manière à ce que le poids total des arêtes choisies soit minimal.

- Implémenter un algorithme permettant de résoudre ce problème. Votre programme doit prendre en entrée, un fichier standardisé. Chaque ligne du fichier contient 3 nombres séparés par un espace, par exemple : 2 8 10. Les deux premiers nombres décrivent l'arête (2 et 8 sont les indices des extrémités de l'arête), le dernier représente le poids 10 de l'arête. En particulier, vous devez vous assurer que le fichier fourni est valide et, dans le cas contraire, informer l'utilisateur.
- Votre programme fournira en sortie un fichier du même type avec, sur chaque ligne 3 nombres séparés par un espace. Ces 3 nombres codent la solution trouvée : les arêtes conservées et leur poids. Ces entrées/sorties standards permettent de tester votre code de façon automatique.
- Dans un second temps, trouver un sous-arbre couvrant de poids minimal qui n'est pas identique au premier (utile en cas de panne ou, pour avoir une solution de secours).
- Enfin, on s'intéresse à la question suivante : Comment maintenir rapidement un MST quand des arêtes ou des sommets sont ajoutés/supprimés dynamiquement, ou quand les poids changent ?
- Pistes d'améliorations : visualisation graphique, animation montrant l'algorithme en action, recherche d'applications variées, développement d'un jeu de données pertinent, cas de graphes orientés, etc.

Sources conseillées :

- [Combinatorial optimization](#), par B. Korte et J. Vygen (chapitre 6) ; accessible depuis une machine connectée au Réseau ULiège.
- Pour la version dynamique du problème, un survol de techniques [Fully Sequential and Distributed Dynamic Algorithms for Minimum Spanning Trees](#) et [Offline algorithms for dynamic minimum spanning tree problems](#).
- Une [application en neurologie](#).